Nombre: Jhostyn Adriano Paca Pérez

Carrera: Electronica y Automatizacion

NRC: 36460

Fecha: 12/10/2026

Materia: Fundamentos de programación

Unidad 1: Ejercicios de programación y algoritmia

Fundamentos de Programación

¡Hola! En esta guía vas a practicar el razonamiento algorítmico paso a paso. Empieza por los problemas 1.1 a 1.9 y luego anímate con los desafíos 1.10 a 1.20. Si algo no sale a la primera, no te preocupes: equivocarse y corregir es parte esencial del proceso. Al final encontrarás una rúbrica para que revises tu propio pensamiento crítico.

Para cada ejercicio, realiza:

1. Análisis del problema
2. Tabla de objetos
3. Seudocódigo
4. Diagrama de flujo
5. Prueba de escritorio

Problema 1.1. Media de dos valores

Desarrolla un programa que ingrese dos números por teclado e imprima su media.

Realiza y completa la tabla de objetos del problema 1.1:

1) Analisis del problema

Entrada: Dos números reales, x y y

Proceso: Media ← (x + y) / 2

Salida: Media

Restricciones: Sin restricciones, solo números reales

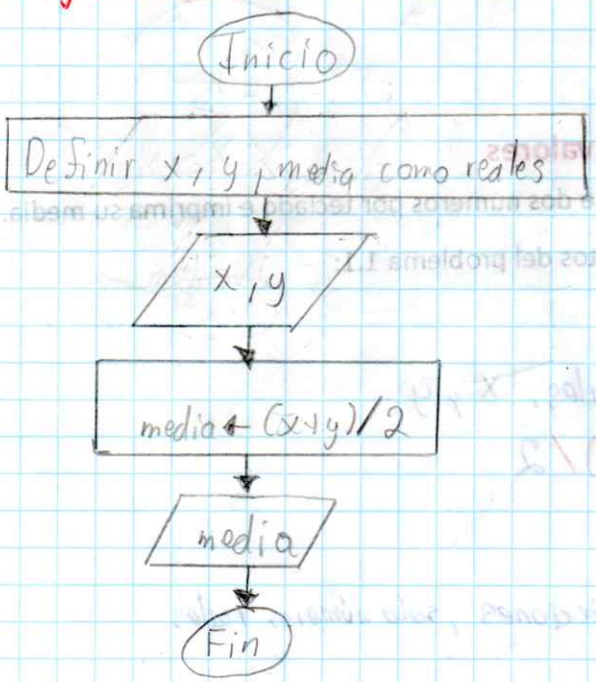
2) Tabla de objetos

Objeto	Nombre	Valor	Tipo
Variable	x	Primer número (6)	Real
Variable	y	Segundo número (2)	Real
Variable	media	Resultado de datos ingresados	Real

3) Pseudocódigo

```
1 Algoritmo Media_de_dos_números
2     Definir x, y, media como Real;
3     Leer x;
4     Leer y;
5     media ← (x+y)/2
6     Escribir media
7 Fin algoritmo
```

4) Diagrama de flujo



5) Prueba de escritorio

Nº fila	x	y	media	salida
3	6			
4		2		
5			(6+2)/2 = 4	
6				4

Problema 1.2. Valor absoluto de x al cubo

Desarrolla un programa que lea un número por teclado y escriba por pantalla $|x|^3$.

1) Analisis del problema

Entrada: Un número real x

Proceso: Si $x < 0$

operar c = (x * -1) * (x * -1) * (x * -1)

Sino

operar c = x * x * x

Salida c

Restricciones Sin restricciones o sólo números reales

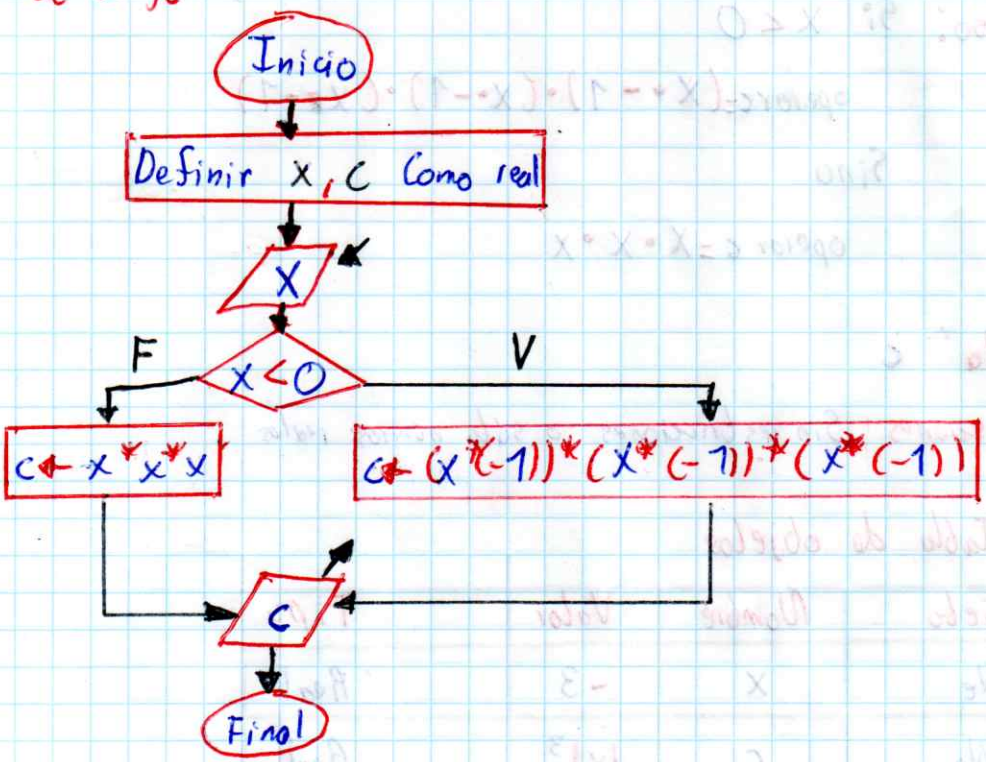
2) Tabla de objetos

Objeto	Nombre	Valor	Tipo
Variable	x	-3	Real
Variable	c	$\|x\|^3$	Real

3) Seudocódigo

```
1  Algoritmo  valor_absoluto_al_cubo
2      Definir  x , c  Como Real;
3      Leer  x;
4      Si  x < 0  entonces
5          c ← (x * (-1)) * (x * (-1)) * (x * (-1));
6      Sino
7          c ← x * x * x;
8      Fin Si;
9      Escribir  c;
10 Fin  algoritmo
```

4) Diagrama de flujo



5) Prueba de escritorio

Nº Filas	x	c	Salida
3	-3		
5		(-3*(-1))*(-3*(-1))*(-3*(-1))	
9			27

Problema 1.3. Divisible

Desarrolla un programa que lea dos números enteros por teclado y determine si el primero es divisible por el segundo. Se mostrará por pantalla el resultado.

> **Pista:** La solución utiliza el operador módulo (%), que devuelve el resto de la división entera entre números enteros. Si x es divisible entre y, el resto debe ser 0.

1) Análisis del problema

Entrada Dos números enteros x, y

Proceso Si x % y ≠ 0

escribir x no es divisible para y

Sino

escribir x es divisible para y

c ← x / y

Salida escribir si es divisible o no

c

Restricción es: y ≠ 0

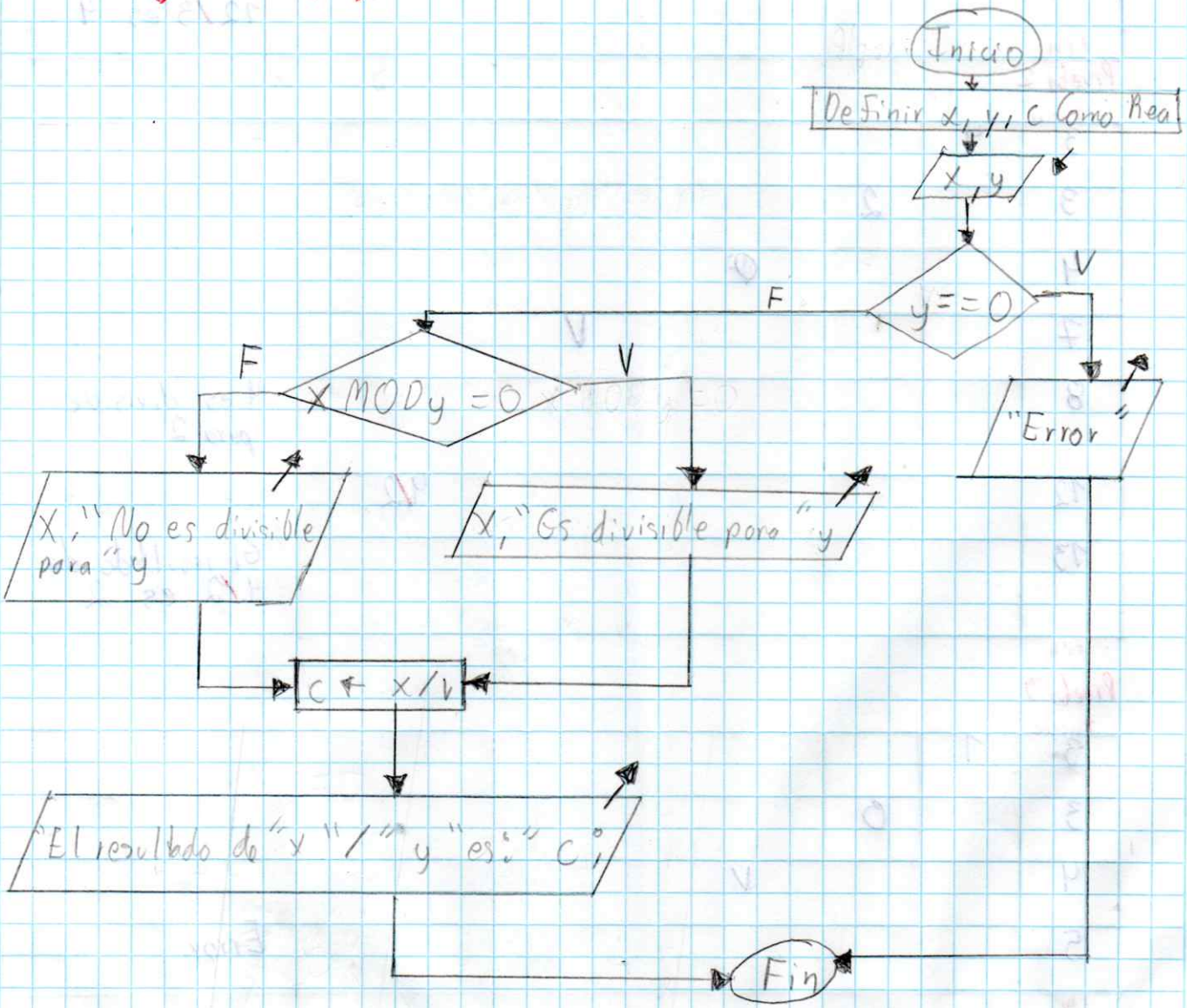
2) Tabla de objetos

Objeto	Nombre	Valor	Tipo
Variable	x	12, 4, 1	Entero
Variable	y	3, 2, 0	Entero
Variable	c	resultado	Real

3) Seudocódigo

```
1  Algoritmo divisibilidad
2     Definir x, y, c Como Real;
3     Leer x, y;
4     Si y == 0;
5        Escribir "Error";
6     Sino
7        Si x % y = 0
8           Escribir x " es divisible para " y;
9        Sino
10          Escribir x " no es divisible para " y;
11       Fin Si
12       c ← x / y;
13       Escribir "El resultado de " x " / " y " es: " c;
14    Fin Si
15 Fin Algoritmo
```

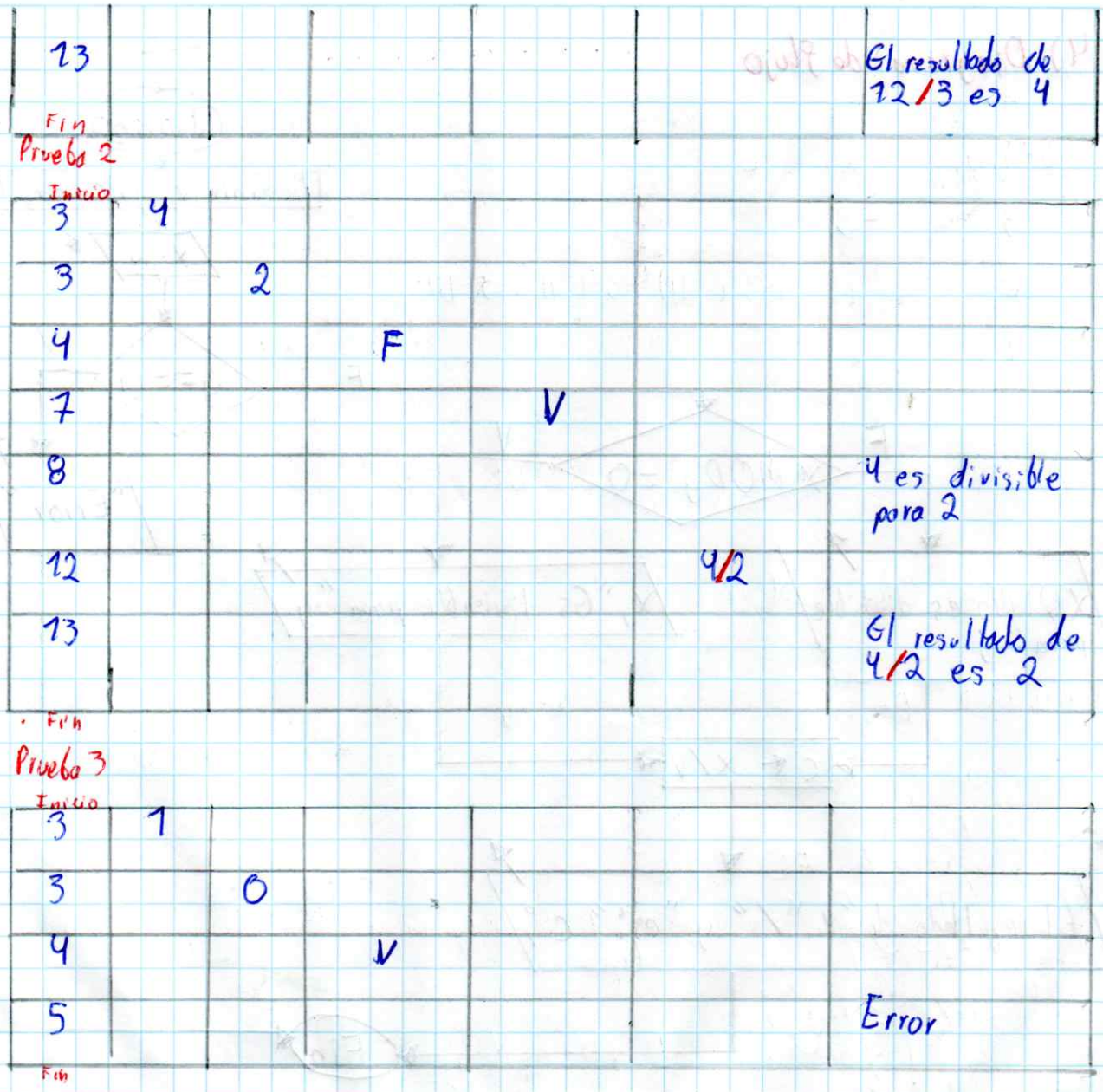
4) Diagrama de flujo



5) Pruebas de escritorio

Nº Fila Inicio	x	y	y==0	x%y=0	c	Salida
3	12					
3		3				
4			F			
7				V		
8						12 es divisible para 3
12					12/3	

ESTILO



Problema 1.4. Intervalo

Desarrolla un programa que lea un número por teclado y determine si pertenece al intervalo [0, 10], indicando el resultado por pantalla.

> **Pista:** El número introducido debe cumplir $x \geq 0$ y $x \leq 10$ (ambas condiciones a la vez) para pertenecer al intervalo. Para establecer estas dos condiciones se utiliza un operador lógico: el operador Y en el diagrama de flujo y && en C. Evita la tentación de escribir $0 \leq x \leq 10$ como una sola condición, porque en C no se evalúa como esperas.

1) Análisis del problema

Entrada Un número entero x

Proceso Si x existe en el intervalo de [0, 10]
escribir x Existe
Sino x no existe

Salida Escribir si x existe o no en el intervalo de [0, 10]

Restricciones: Solo permite numeros enteros

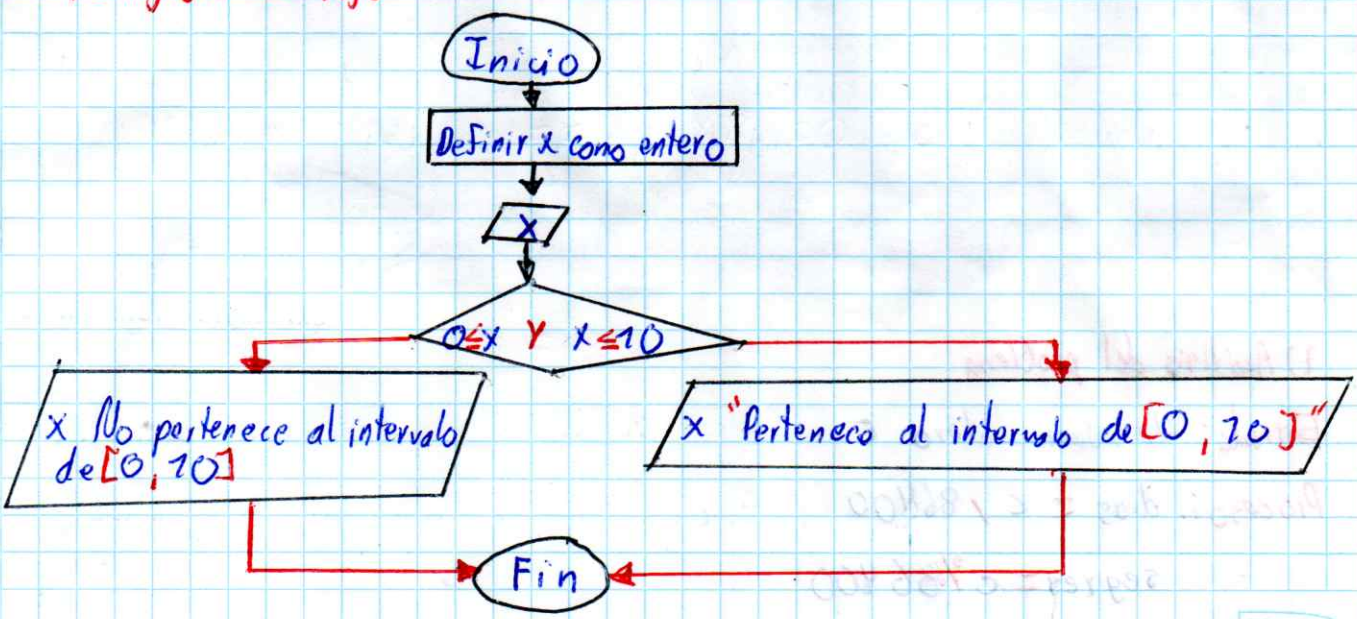
2) Tabla de objetos

Objeto	Nombre	Valor	Tipo
Variable	X	-1,5,11	Entero

3) Seudo código

```
1 Algoritmo intervalo
2     Definir x Como entero;
3     leer x;
4     Si 0≤x Y x≤10 entonces
5         Escribir x "Pertenece al intervalo de [0, 10]";
6     Si no
7         Escribir x "No pertenece al intervalo de [0 ,10]";
8     Fin Si
9 Fin Algoritmo
```

4) Diagrama de Flujo



5) Pruebas de escritorio

Nº Fila	x	0 ≤ x Y x ≤ 10	Salida
Inicio 3	-1	F	-1 No pertenece al intervalo de [0,10]
Fin			
Inicio 3	5		
4		V	
5			5 Pertenece al intervalo de [0,10]
Fin			
Inicio 3	11		
4		F	
5			11 No pertenece al intervalo de [0,10]
Fin			

Problema 1.5. Conversión de unidades de tiempo

Desarrolla un programa que lea por teclado un valor entero c, interpretándolo como el número de segundos que dura un evento, y calcule y muestre por pantalla cuántos días, horas, minutos y segundos representa.

Para calcular los días, horas, minutos y segundos se utilizan la división entera y el módulo (resto de una división, operador %). Sea c el número de segundos a convertir. Como un día tiene 24 horas, una hora tiene 60 minutos y un minuto tiene 60 segundos, el cociente entero c / (24 · 60 · 60) es el número de días completos contenidos en c. Asimismo, c % (24 · 60 · 60) es el número de segundos restantes tras descontar esos días. Se procede de igual manera para obtener las horas (1 hora = 60 · 60 segundos) y los minutos (1 minuto = 60 segundos).

Ejemplo: supón que quieres convertir c = 10 000 segundos. Como 10 000 / (24 · 60 · 60) = 0, hay 0 días, porque 10 000 segundos es menos que un día. Análogamente, 10 000 / (60 · 60) = 2 horas. Si a 10 000 segundos les quitas esas 2 horas, quedan 10 000 % (60 · 60) = 2 800 segundos, que son algo más de 46 minutos, cantidad que se obtiene con 2 800 / 60 = 46 minutos. Finalmente, 2 800 % 60 = 40 segundos. Por lo tanto, 10 000 segundos equivalen a 0 días, 2 horas, 46 minutos y 40 segundos.

1) Análisis del problema

Entrada: Un número entero c

Proceso: dias = c / 86400

c = c % 86400

horas = c/3600;
c = c%3600;
minutos = c/60
c = c%60

Salida: Dias
horas
minutos
segundos

Restricciones: c no puede ser menor a 0
c debe ser entero

2) Tabla de objetos

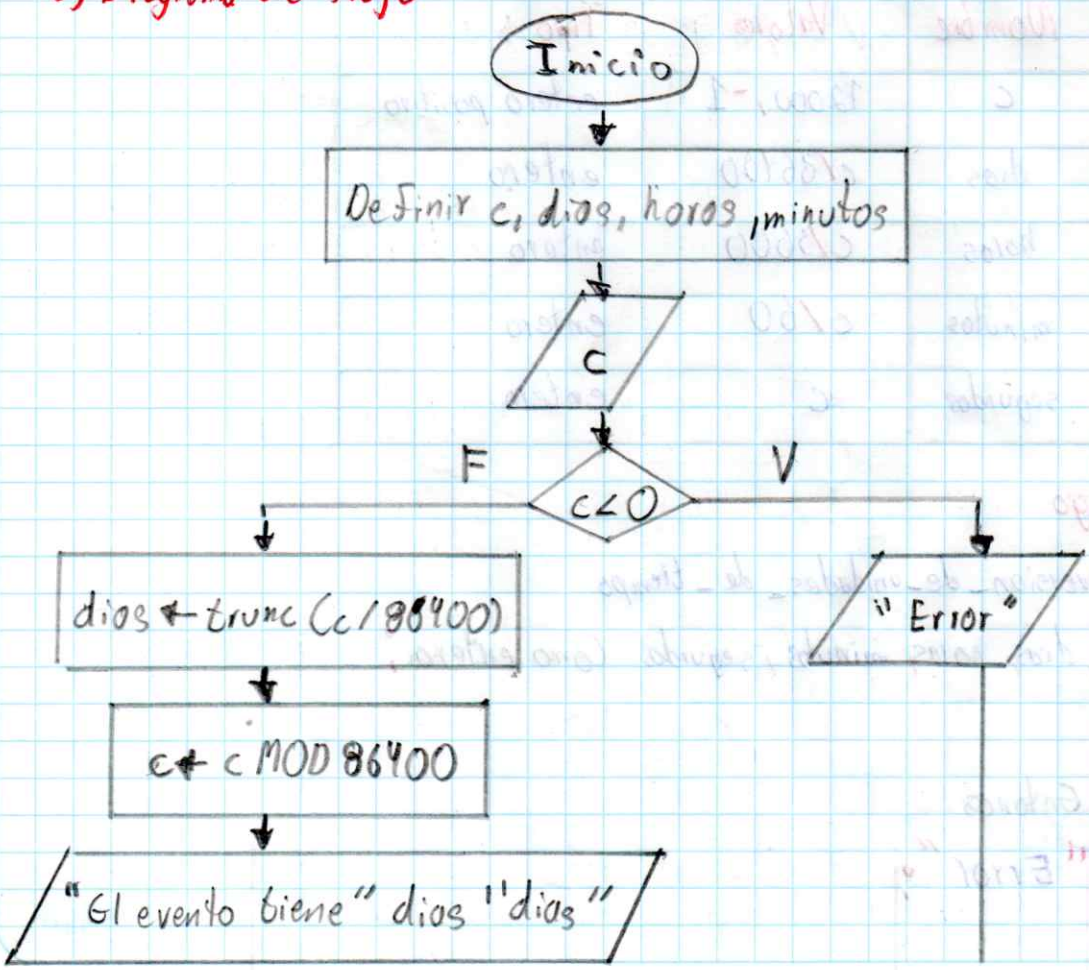
Objeto	Nombre	Valores	Tipo
Variable	c	12000, -1	entero positivo
Variable	dias	c/86400	entero
Variable	horas	c/3600	entero
Variable	minutos	c/60	entero
Variable	segundos	c	entero

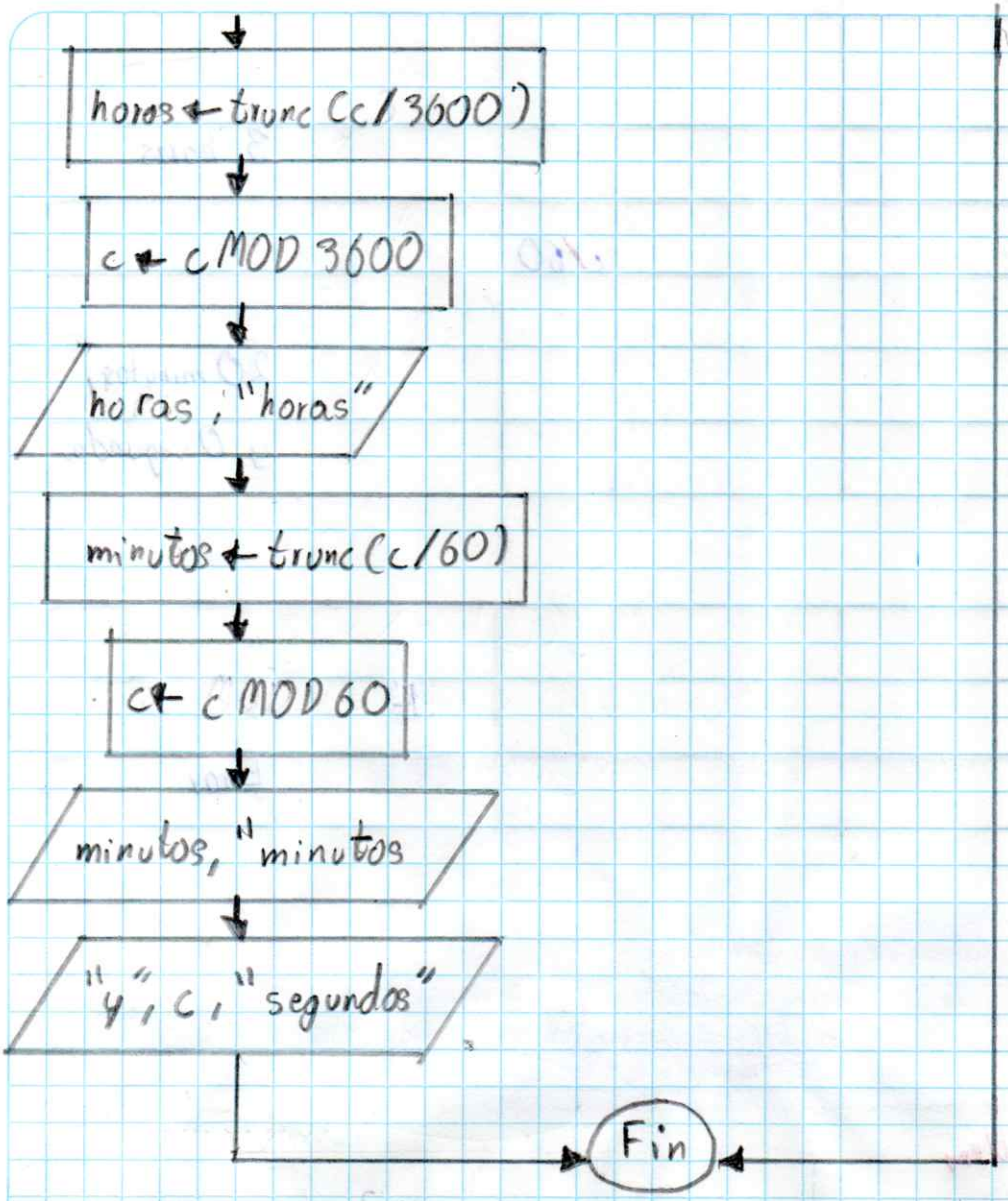
3) Seudocódigo

```
1 Algoritmo conversion_de_unidades_de_tiempo
2   Definir c, dias, horas, minutos, segundos Como entero;
3   Leer c;
4   Si c<0 Entonces
5     Escribir "Error";
6   Sino
```

```
7    dias ← trunc(c/86400);
8    c ← c %. 86400;
9    Escribir "El evento tiene" dias "días";
10   horas ← trunc(c/3600);
11   c ← c/3600
12   Escribir "horas" "horas";
13   minutos ← trunc(c/60);
14   c ← c %. 60
15   Escribir minutos "minutos";
16   Escribir "y" c "segundos";
17 FinSi
18 Fin Algoritmo
```

4) Diagrama de flujo





5) Pruebas de escritorio

10ºFila 1ra prueba	c	dias	horas	minutos	c<0	Salida
3	12000					
4					F	
7		c/86400				
8	c%86400					
9						El evento tiene 0 dias
10			c/3600			

ESTILO

11	c/.3600					
12						3 horas
13				c/60		
14	c%60					
15						20 minutos
16						y 0 segundos

2da prueba

3	-1					
4					V	
5						Error

Problema 1.6. Máximo, mínimo y media de alturas

Desarrolla un programa que lea por teclado un número indeterminado de alturas y deje de leer cuando se introduzca una negativa. El programa debe calcular e imprimir la media, el máximo y el mínimo de las alturas introducidas.

1) Analisis del problema

Entrada: n números reales

Proceso: Repetir n veces las veces que repetira el bucle

Salida: los diferentes alturas establecidas

Restricciones: n ≠ 0 y números negativos

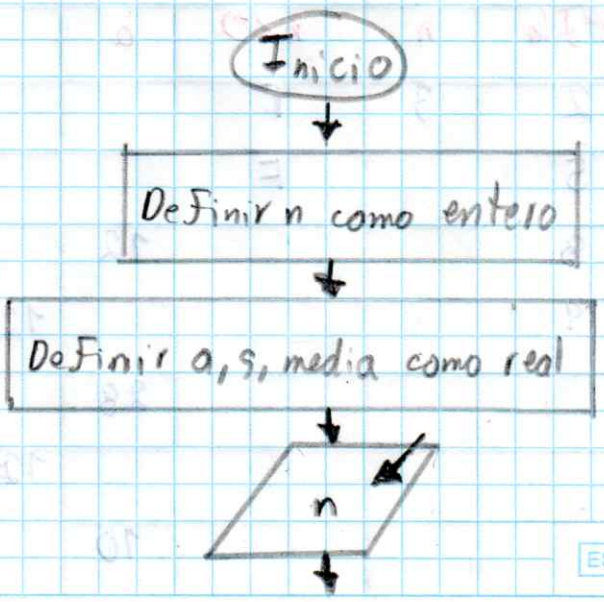
2) Tabla de objetos

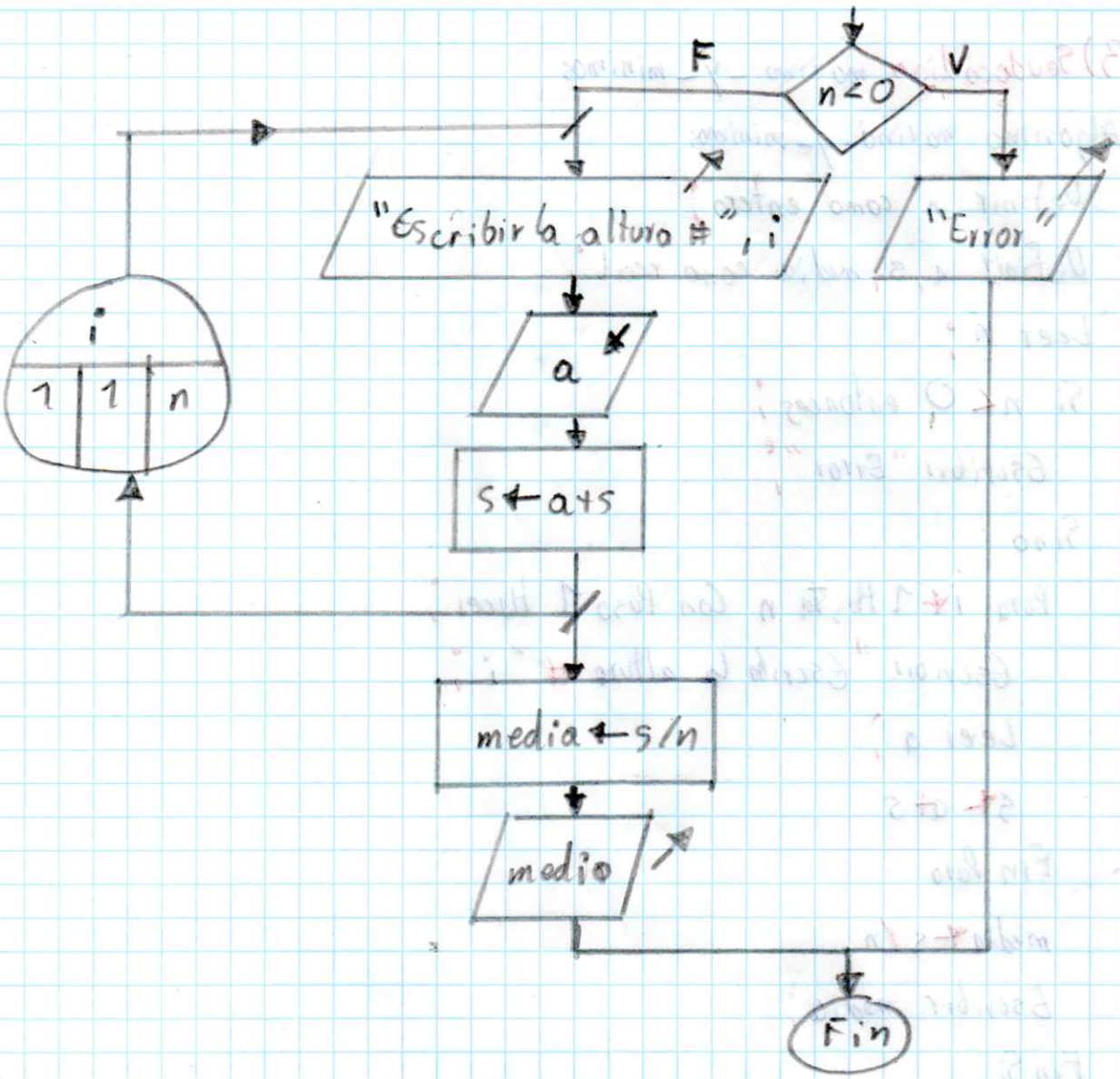
Objeto	Nombre	Valor	Tipo
Variable	i	3, -1	entero
Variable	a ...	alturas	real
Variable	s	sumas de a	real
Variable	media	media de s	real

3) Seudocódigo máximo_y_minimos

```
1  Algoritmo máximo_y_minimos
2    Definir n como entero;
3    Definir a, s, media como real;
4    Leer n;
5    Si n<0 entonces;
6      Escribir "Error";
7    Sino
8      Para i←1 Hasta n Con Paso 1 Hacer;
9        Escribir "Escriba la altura #" i;
10       Leer a;
11       s←a+s
12     Fin Para
13     media←s/n
14     Escribir media;
15   Fin Si
16 Fin Algoritmo
```

4) Diagrama de Flujo





5) Pruebas de escritorio

Prueba #1

Nº Fila	n	n<0	a	s	media	Salida
2	3					
5		F				
8			12			
9				12+0		
			28			
				12+28		
			14			

40+14

5413

14

18 es la media

El máximo es 28 y el mínimo es 12

Prueba #2

2	-1	
5		V
6		Error

Problema 1.7. Raíz cuadrada entera

Desarrolla un programa que calcule la raíz cuadrada entera por defecto de un número N positivo dado, que se define como el mayor número entero $R > 0$ tal que $R^2 \leq N$.

> **Pista:** Dada una variable entera $i = 1$, itera incrementando su valor de uno en uno mientras $i^2 \leq N$. La aproximación por defecto de la raíz es $R = i - 1$.

1) Analisis del problema

Entrada: Un número entero x

Proceso: Iterar x veces para sumar i ← i+1

Salida: La raiz cuadrada entera por defecto z

Restricción: x > 0

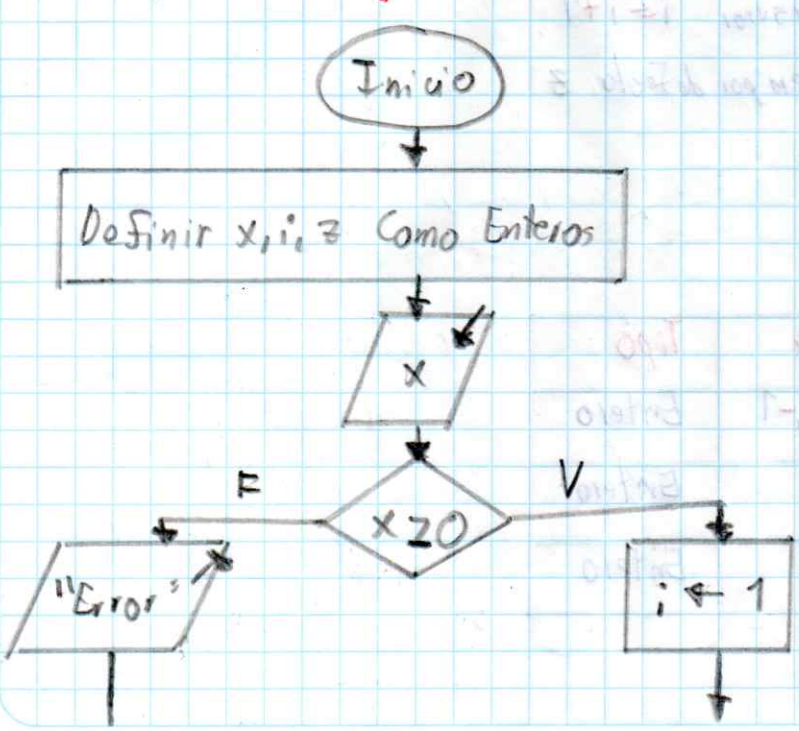
2) Tabla de objetos

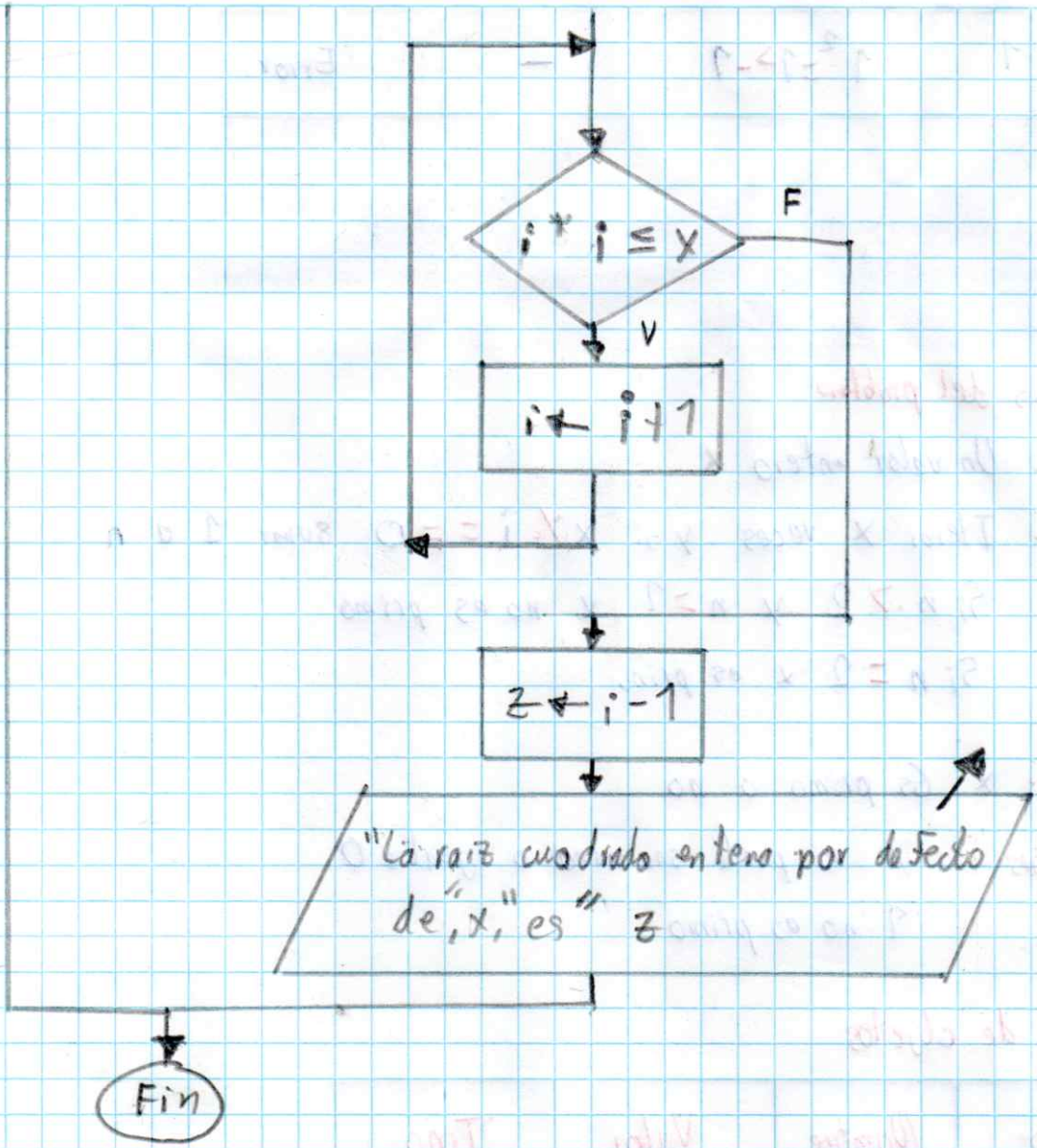
Objeto	Nombre	Valor	Tipo
Var	x	9, 8, -1	Entero
Var	i		Entero
Var			Entero

3) Seudo código

```
1  Algoritmo Raiz_cuadrado_entera
2    Definir x, i, z Como entero
3    Leer x;
4    Si x ≥ 0 entonces
5      i ← 1
6      Mientras i * 1 ≤ x Hacer
7        i ← i + 1
8      Fin Mientras
9      z ← i - 1
10     Escribir "La raiz cuadrada entera por defecto de ", x, " es: ", z
11   Si No
12     Escribir "Error";
13   FinSi
14 Fin Algoritmo
```

4) Diagrama de Flujo





5) Pruebas de escritorio

X	Paso	i	Condición $i^2 \leq x$	Acción	Resultado parcial
20	Inicio	1	$i^2 = 1 \leq 20$	Se suma i	i = 2
	Iteración 2	2	$2^2 = 4 \leq 20$	Se suma i	i = 3
	Iteración 3	3	$3^2 = 9 \leq 20$	Se suma i	i = 4
	Iteración 4	4	$4^2 = 16 \leq 20$	Se suma i	i = 5
	Iteración 5	5	$5^2 = 25 \not\leq 20$	No se suma nada	i = 5
	Final			z = i - 1	z = 4

Prueba #2 -1	1	$1^2 = 1 > -1$	—	Error

Problema 1.8. Número primo

Desarrolla un programa que ingrese por teclado un número entero mayor que 1 y determine si es primo o no.

1) Análisis del problema

Entrada: Un valor entero x

Proceso: Iterar x veces y si x % i == 0 sumar 1 a n

Si n > 2 y n = 1 x no es primo

Si n = 2 x es primo

Salida: x Es primo o no

Restricciones: x no puede ser menor o igual a 0

1 no es primo

2) Tabla de objetos

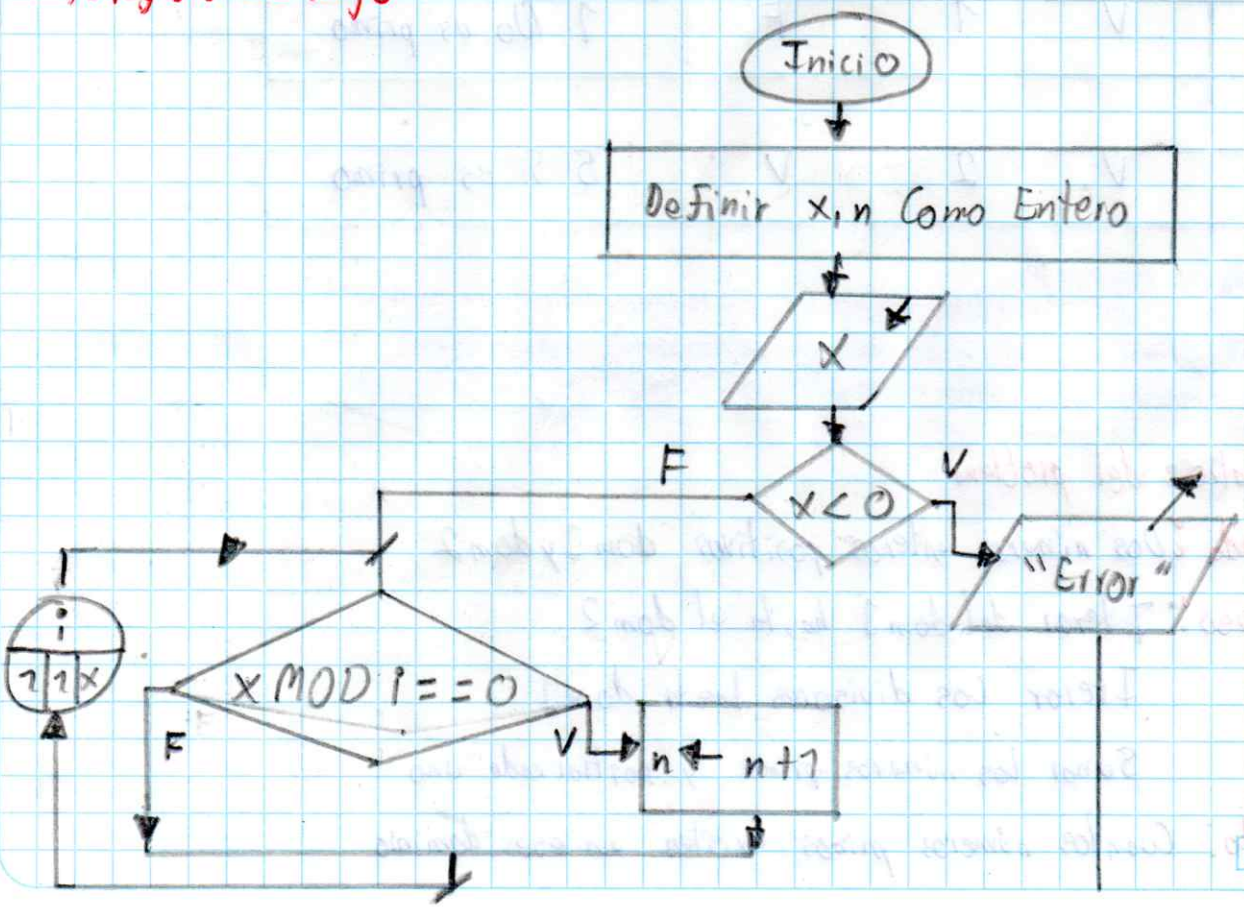
Objeto	Nombre	Valor	Tipo
Var	x	0 ; 1,5	Entero
Iteración	i		Entero
Var	n		Entero

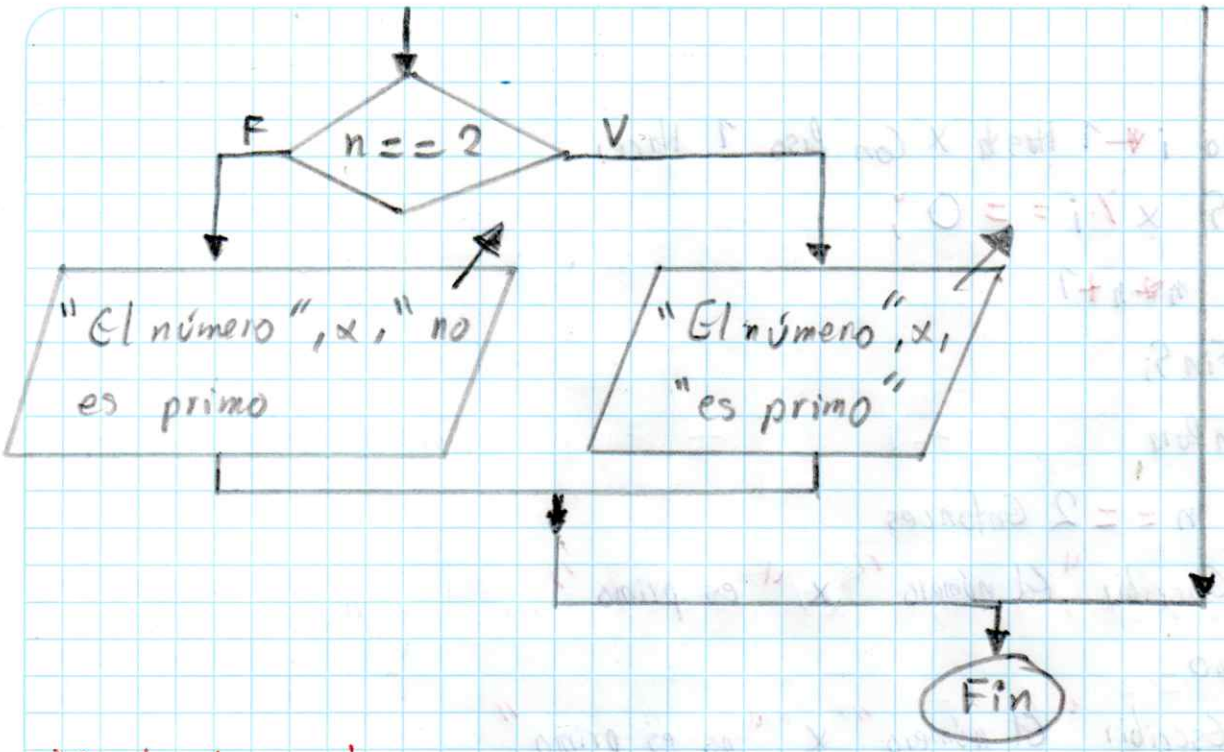
3) Seudocódigo

```
1 Algoritmo número_primo
2   Definir x, n Como entero;
3   Leer x;
4   Si x < 0 Entonces;
5     Escribir "Error";
```

```
6  Sino
7    Para i ← 1 Hasta x Con Paso 1 Hacer
8      Si x % i = = 0 ;
9        n ← n + 1
10     Fin Si
11   Fin Para
12   Si n = = 2 Entonces
13     Escribir "El número" x "es primo"
14   Sino
15     Escribir "El número" x "no es primo"
16   Fin Si
17 Fin Si
18 Fin Algoritmo
```

4) Diagrama de Flujo





5) Pruebas de escritorio

x	x<0	n	n==2	Salida
0	F			Error
1	V	1	F	1 No es primo
5	V	2	V	5 Si es primo

Problema 1.9. Números primos comprendidos entre dos números dados

Desarrolla un programa que muestre por pantalla todos los números primos comprendidos entre dos números enteros introducidos por teclado.

1) Analisis del problema

Entrada: Dos números enteros positivos dom 1 y dom 2

Proceso: Iterar del dom 1 hasta el dom 2

Iterar los divisores hasta dom 1

Sumar los números primos y mostrar cada uno

Salida: Cuantos números primos existen en ese dominio

Mostrar todos los números primos en ese dominio

Restricciones: - $0 \leq$ dom 1, dom 2 && enteros positivos

2) Tabla de objetos

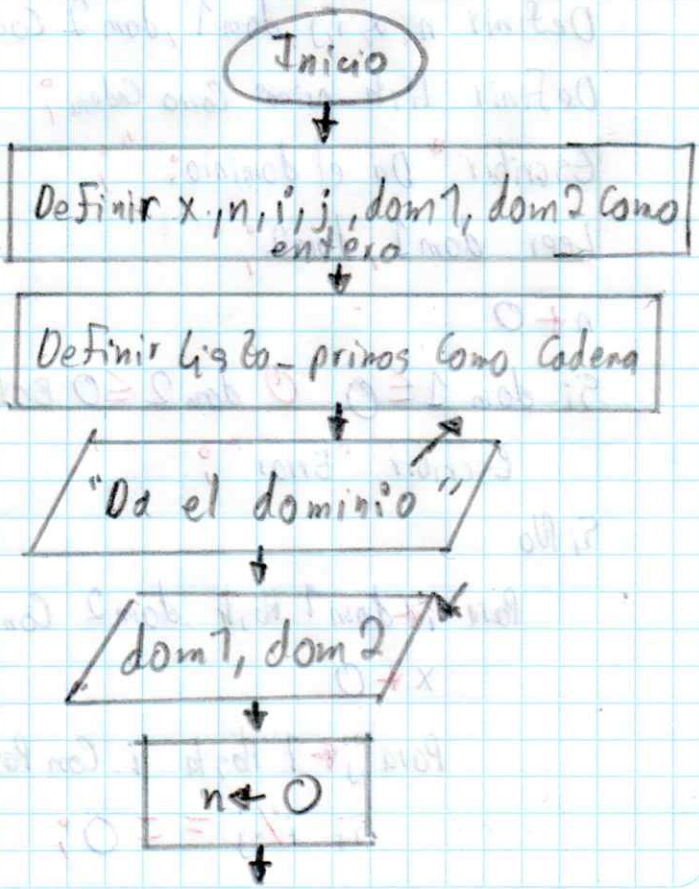
Objeto	Nombre	Valor	Tipo
Var	n	0	entero
Var	x	0	entero
Var	i	dom1	entero
Var	j	1	entero
Var	dom1	1, 0	entero
Var	dom2	8, -5	entero
Var	Lisb_primos		cadena

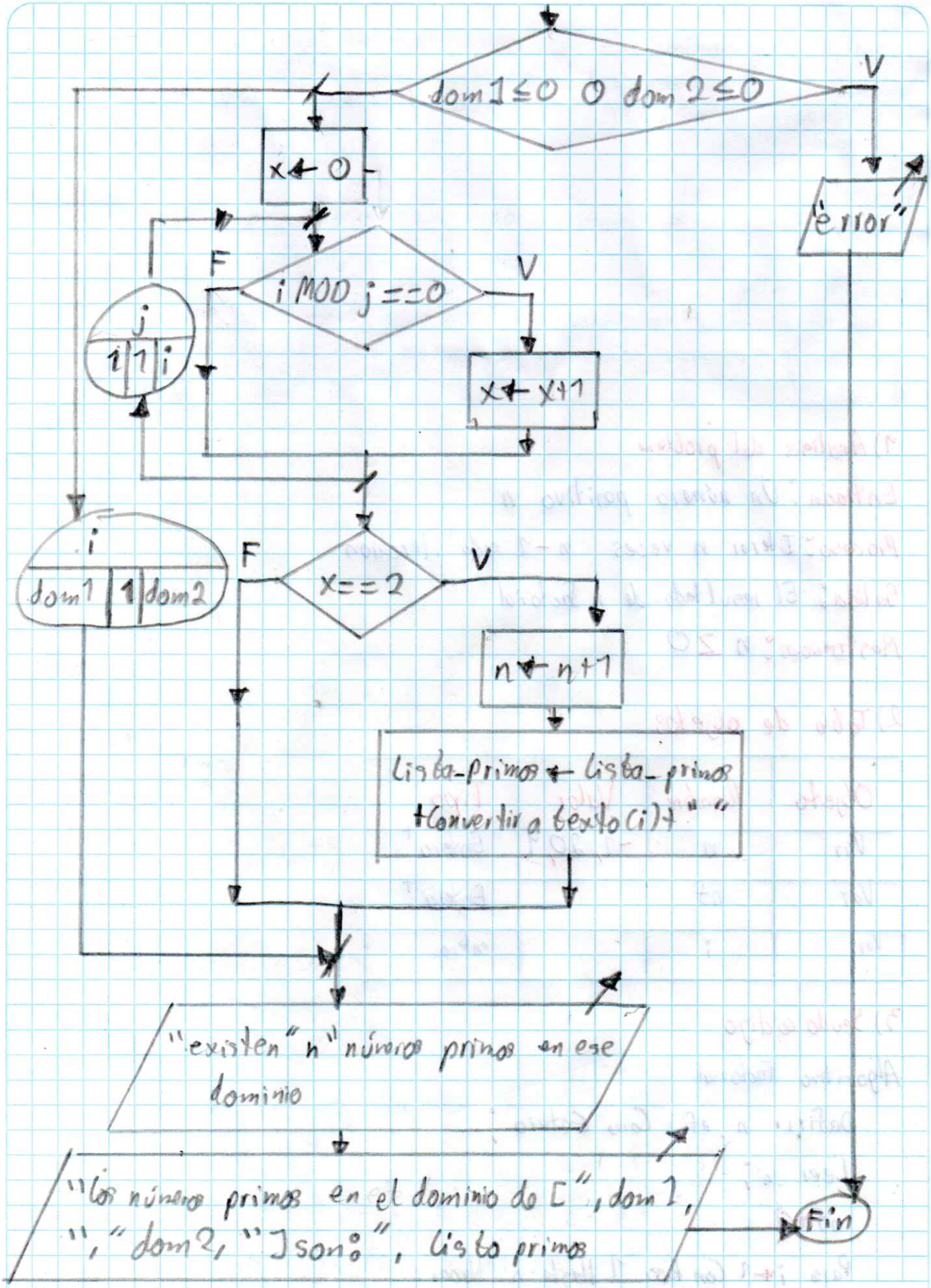
3) Seudocódigo

```
Algoritmo Numeros_primos_comprendidos_entre_dos_números_dados
    Definir n, x, i, j, dom1, dom2 Como Entero;
    Definir lisb_primos Como Cadena;
    Escribir "Da el dominio: ";
    Leer dom1, dom2;
    n ← 0
    Si dom1 ≤ 0 O dom2 ≤ 0 Entonces
        Escribir "Error";
    Si No
        Para i ← dom1 Hasta dom2 Con Paso 1 Hacer
            x ← 0
            Para j ← 1 Hasta i Con Paso 1 Hacer
                Si i % j = = 0;
```

```
            x ← x + 1;
        FinSi
    FinPara
    Si x = = 2 Entonces
        n = n + 1;
        lista_primos ← lista_primos + ConvertirATexto(i) + " ";
    FinSi
FinPara
Escribir "Existen " n " numeros primos en ese dominio";
Escribir "Los números primos en el dominio de [", dom1, ", ", dom2,
    " ] son: ", lista_primos
FinSi
Fin Algoritmo
```

4) Diagrama de flujo




dom1≤0 O dom2≤0
V
"error"
x ← 0
F
i MOD j == 0
V
j
1 | 1 | i
x ← x+1
i
dom1 | 1 | dom2
F
x == 2
V
n ← n+1
lista_primos ← lista_primos + convertir a texto(i) + " "
"existen" n "números primos en ese dominio"
"los números primos en el dominio de [", dom1, ", ", dom2, "] son:", lista primos
Fin

Desafíos de algoritmia

Estos once ejercicios suben poco a poco de dificultad. En cada uno encontrarás una pregunta "Para pensar": respóndela por escrito, porque la rúbrica final valora tu capacidad de justificar y evaluar tus decisiones.

Problema 1.10. Factorial

Desarrolla un programa que lea un número entero $n \geq 0$ y calcule $n! = 1 \cdot 2 \cdot \ldots \cdot n$, sabiendo que $0! = 1$.

Ejemplo: $n = 5 \rightarrow 120$.

> **Pista:** Usa un acumulador que inicie en 1 y se multiplique en cada vuelta del ciclo.

> **Para pensar:** ¿Qué ocurre si el usuario ingresa un número negativo? ¿Y con n = 20 en una variable de tipo int?

1) Analisis del problema

Entrada: Un número positivo n

Proceso: Iterar n veces n-1 cada iteracion

Salida: El resultado de n factorial

Restricciones: $n \geq 0$

2) Tabla de objetos

Objeto	Nombre	Valor	Tipo
Var	n	-1, 20, 4	Entero+
Var	nf		Entero+
Var	i		entero

3) Seudo código

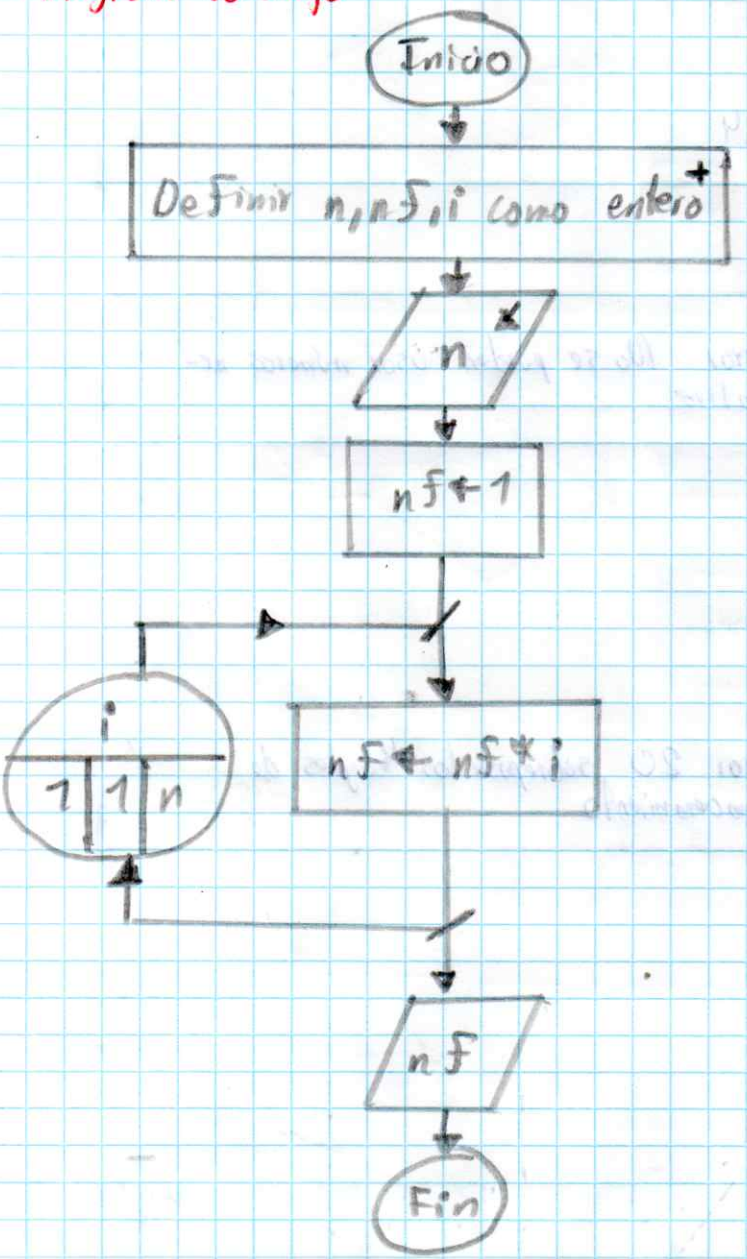
```
Algoritmo Factorial
    Definir n, nf, i Como Entero;
    Leer n;
    nf ← 1
    Para i ← 1 Con Paso 1 Hasta n Hacer
        nf ← nf * i
```

FinPara

Escribir nf;

FinAlgoritmo

4) Diagrama de Flujo



5) Pruebas de escritorio

Entrada	nf	Salida
4	nf=4.1 nf=3.2 nf=2.4	24

-7	nf=1.1	Error - No se pueden usar números negativo en los factoriales
20	nf=1.1 =1.2 =2.3 ⋮ =19.20	Error 20 sobrepaso los 4bytes de almacenamiento